CHUKA UNIVERSITY
Center for Data Analytics & Modeling
Website: https://cdam.chuka.ac.ke

Session 6: Correlation and Regression Analysis

Complied by D K. Muriithi

Historical Origin of the Term Regression

The term regression was introduced by Francis Galton. In a famous paper, Galton foun that, although there was a tendency for tall parents to have tall children and for short paren s to have short children, the average height of children born of parents of a given hei ht tended to move or "regress" toward the average height in the population as a whole. In other words, the height of the children of unusually tall or unusually short parents ten s to move toward the average height of the population. Galton's law of universal regressi n was confirmed by his friend Karl Pearson, who collected more than a thousand rec rds of heights of members of family groups.2 He found that the average height of sons of a group of tall fathers was less than their fathers' height and the average height of sons o a group of short fathers was greater than their fathers' height, thus "regressing" tall and hort sons alike toward the average height of all men. In the words of Galton, this was "re ression to mediocrity."

Correlational Analysis is a statistical technique used to measure the strength and direction of the relationship between two or more variables.

Purpose

- Determine whether variables are related.
- Assess the strength of the relationship.
- Identify whether the relationship is positive, negative, or absent.

Types of Correlation

- **Positive correlation**: As one variable increases, the other also increases.
  - Example: Study hours and exam scores.
- **Negative correlation**: As one variable increases, the other decreases.
  - Example: Exercise frequency and body fat percentage.
- **Zero correlation**: No relationship exists between the variables.

Common Correlation Coefficients

- **Pearson's correlation (r)**: For continuous, normally distributed variables.
- **Spearman's rank correlation (ρ)**: For ordinal data or non-normal distributions.

- **Kendall's Tau (τ)**: For ordinal data and small sample sizes.

Interpretation of Pearson's r

Correlation Coefficient (r)	Interpretation
0.00–0.19	Very weak
0.20–0.39	Weak
0.40–0.59	Moderate
0.60–0.79	Strong
0.80–1.00	Very strong

(The same applies for negative values, indicating an inverse relationship.)

Important Note

Correlation does not imply causation. A strong correlation between two variables does not necessarily mean that one causes the other.

Example: A researcher may use correlational analysis to examine the relationship between malaria incidence and rainfall levels across counties. A positive correlation would suggest that higher rainfall is associated with increased malaria cases.

Pearson Correlational Analysis

Calculate Pearson correlation

In [1]:
```
# install necessary libraries
#!pip install Image
#!pip install Pillow
```

In [2]:
```
from PIL import Image

# Open the image file
img = Image.open('capture.png')
img
```

Out[2]:

$$r = \frac{n\sum xy - \sum x \sum y}{\sqrt{n\sum x^2 - (\sum x)^2}\sqrt{n\sum y^2 - (\sum y)^2}}$$

In [3]:
```
import pandas as pd
import numpy as np
from scipy.stats import pearsonr, spearmanr

# Sample data
```

```
data = pd.DataFrame({
    'Age': [33, 40, 35, 43, 44, 23, 34, 54, 33],
    'Weight': [67, 77, 69, 71, 70, 65, 65, 89, 75]
})
data
```

Out[3]:

	Age	Weight
0	33	67
1	40	77
2	35	69
3	43	71
4	44	70
5	23	65
6	34	65
7	54	89
8	33	75

In [4]:
```
from scipy.stats import pearsonr

r, p_value = pearsonr(data["Age"], data["Weight"])
print(f"Pearson correlation coefficient:{r:.3f}")
print(f"p-value:{p_value:.3f}")
```

```
Pearson correlation coefficient:0.781
p-value:0.013
```

Calculate Spearman correlation

In [5]:
```
spearman_corr, spearman_p = spearmanr(data['Age'], data['Weight'])
print(f"Spearman Correlation: {spearman_corr:.3f}, p-value: {spearman_p:.3f}")
```

```
Spearman Correlation: 0.647, p-value: 0.060
```

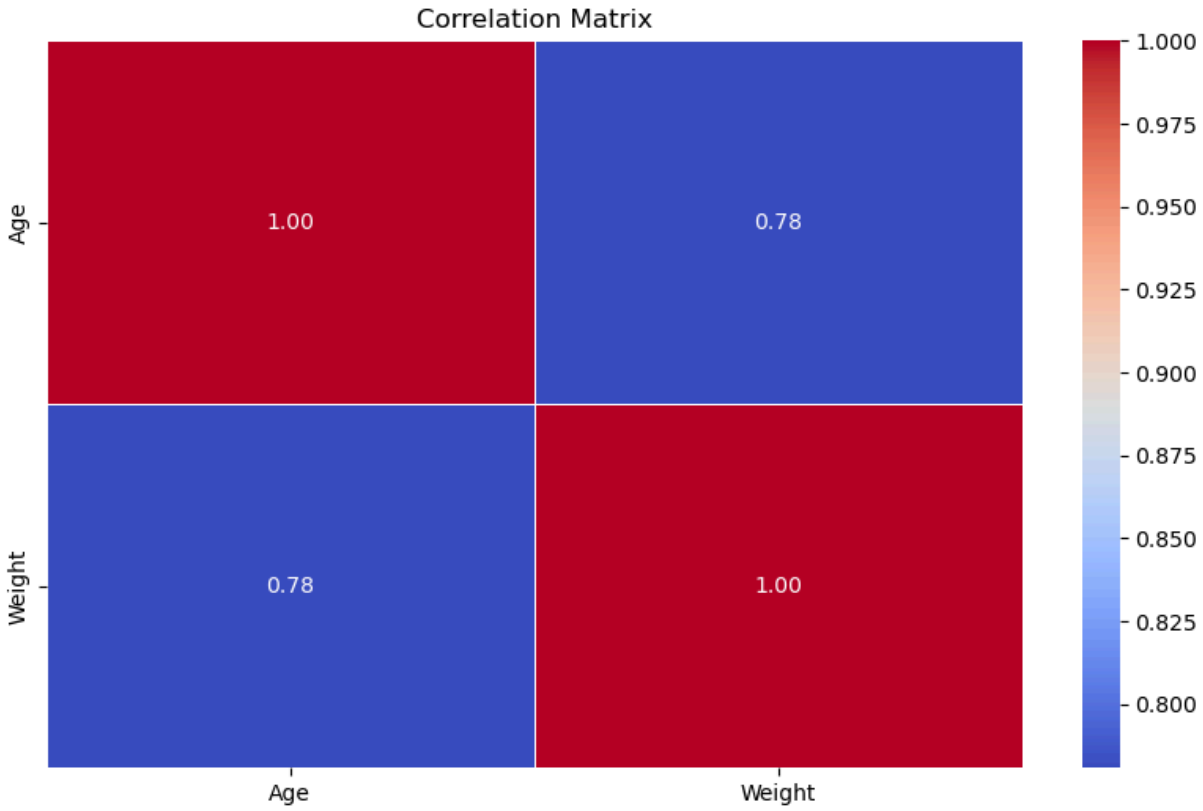
In [6]:
```
import pandas as pd

# Calculate correlation
r = data['Age'].corr(data['Weight'])
print(f"Pearson correlation coefficient:{r:.4f}")
```

```
Pearson correlation coefficient:0.7808
```

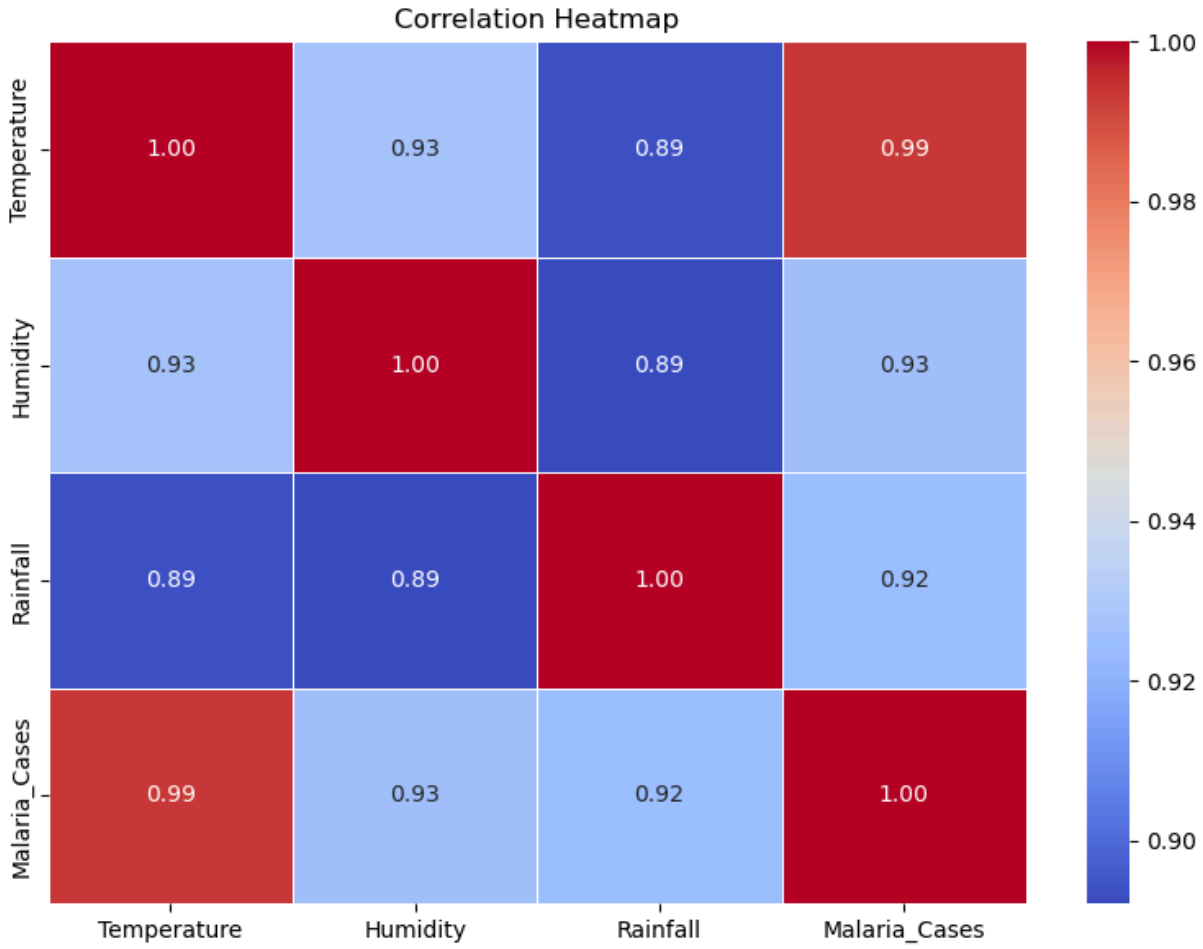
Correlation Heatmap

In [7]:
```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt  # ✅ this fixes the error
```

```
# Assuming `data` is your DataFrame
corr_matrix = data.select_dtypes(include=[np.number]).corr()

plt.figure(figsize=(10, 6))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt='.2f', linewidths=0.5)
plt.title('Correlation Matrix')
plt.show()
```



In [8]:
```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Sample DataFrame (replace this with your actual dataset)
data1 = pd.DataFrame({
    'Temperature': [28, 30, 29, 32, 31],
    'Humidity': [60, 65, 58, 70, 68],
    'Rainfall': [12, 15, 13, 18, 20],
    'Malaria_Cases': [34, 45, 40, 60, 55]
})

# Compute correlation matrix
corr_matrix = data1.corr()

# Plot heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt='.2f', linewidths=0.5)
plt.title('Correlation Heatmap')
plt.tight_layout()
plt.show()
```



Covariance

In [9]:
```
img = Image.open('capture2.png')
img
```

Out[9]:

$$r_{xy} = \frac{Cov(x,y)}{S_x S_y}$$

Regression analysis

Regression analysis is a powerful statistical method used to model, quantify, and analyze the relationship between a dependent variable (the outcome you want to predict) and one or more independent variables (the predictors or factors that influence it)

Core Purposes

Predicting trends: Estimating future outcomes based on historical data patterns.

Measuring impact: Determining how strongly specific factors influence the main outcome.

Optimizing processes: Identifying which variables can be ignored and which are critical.

Common Types

Simple Linear Regression: Uses a single independent variable to predict a continuous outcome by fitting a straight line through the data points (e.g., predicting a house price based solely on its square footage).

Multiple Linear Regression: Uses two or more independent variables to predict a continuous outcome (e.g., predicting a house price based on square footage, number of bedrooms, and neighborhood crime rates).

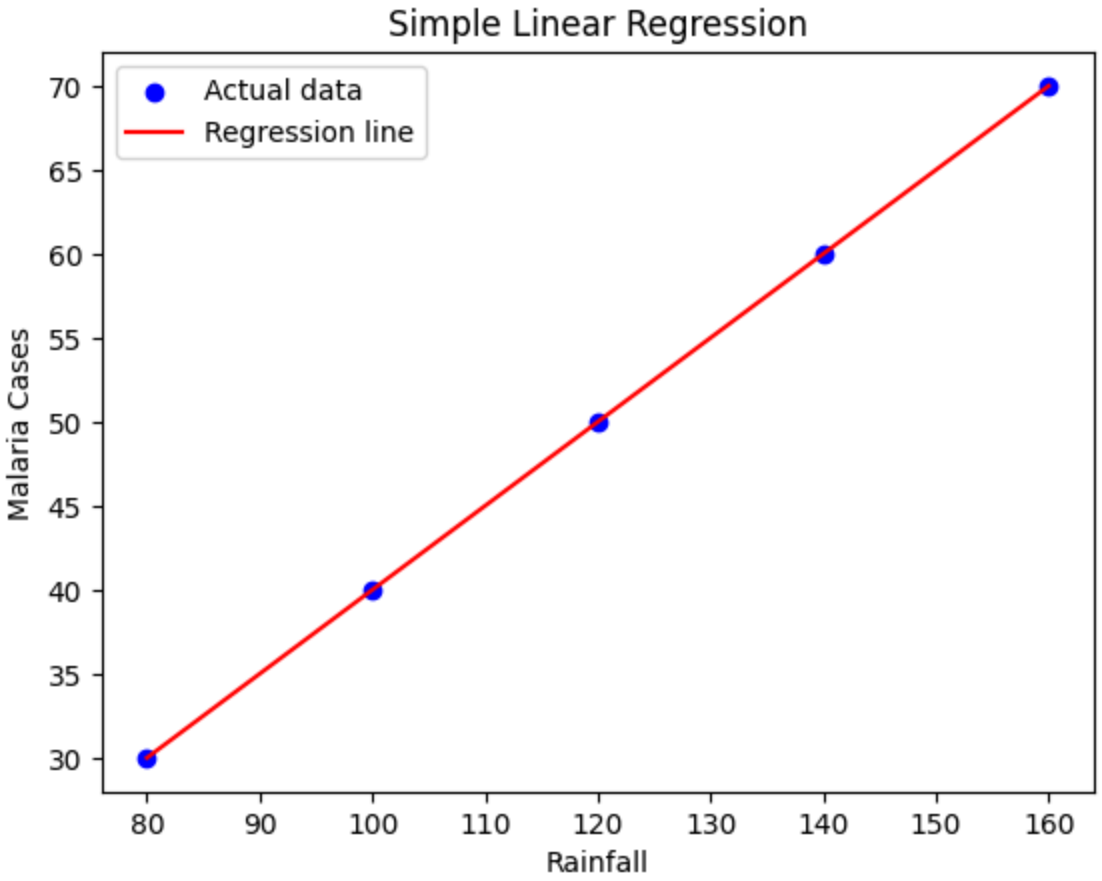
Logistic Regression: Used when the outcome variable is categorical rather than a continuous number, predicting the probability of an event occurring (e.g., determining whether a transaction is "fraudulent" or "legitimate")

Simple Linear Regression Model

In [1]:
```python
import pandas as pd
from sklearn.linear_model import LinearRegression

# Sample data
data2 = pd.DataFrame({
    'Rainfall': [80, 100, 120, 140, 160],
    'Malaria_Cases': [30, 40, 50, 60, 70]
})

# Define X and y
X = data2[['Rainfall']]  # 2D for sklearn
y = data2['Malaria_Cases']

# Initialize and fit the model
model = LinearRegression()
model.fit(X, y)

# Print coefficient and intercept
print("Intercept:", model.intercept_)
print("Slope (Rainfall):", model.coef_[0])

# Predict malaria cases
y_pred = model.predict(X)
print("Predicted values:", y_pred)
```

```
Intercept: -10.0
Slope (Rainfall): 0.5
Predicted values: [30. 40. 50. 60. 70.]
```

In [2]:
```
import matplotlib.pyplot as plt

plt.scatter(X, y, color='blue', label='Actual data')
plt.plot(X, y_pred, color='red', label='Regression line')
plt.xlabel('Rainfall')
plt.ylabel('Malaria Cases')
plt.title('Simple Linear Regression')
plt.legend()
plt.show()
```



In [3]:
```
import pandas as pd
import numpy as np
from scipy.stats import pearsonr, spearmanr

# Sample data
data = pd.DataFrame({
    'Age': [33, 40, 35, 43, 44, 23, 34, 54, 33],
    'Weight': [67, 77, 69, 71, 70, 65, 65, 89, 75]
})
```

In [4]:
```
import statsmodels.api as sm
data
# Define independent and dependent variables
X = sm.add_constant(data['Age'])  # Adds a constant term to the predictor
y = data['Weight']

# Fit the model
```

```
model = sm.OLS(y, X).fit()

# Print summary of the model
print(model.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                 Weight   R-squared:                       0.610
Model:                            OLS   Adj. R-squared:                  0.554
Method:                 Least Squares   F-statistic:                     10.94
Date:                Fri, 03 Jul 2026   Prob (F-statistic):             0.0130
Time:                        17:05:43   Log-Likelihood:                -26.240
No. Observations:                   9   AIC:                             56.48
Df Residuals:                       7   BIC:                             56.87
Df Model:                           1
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const         46.6661      7.845      5.949      0.001      28.116      65.216
Age            0.6726      0.203      3.307      0.013       0.192       1.154
==============================================================================
Omnibus:                        2.454   Durbin-Watson:                   1.743
Prob(Omnibus):                  0.293   Jarque-Bera (JB):                0.862
Skew:                           0.087   Prob(JB):                        0.650
Kurtosis:                       1.494   Cond. No.                         179.
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly spe
cified.
```

Extracting coefficients and R-squared

In [5]:
```
coefficients = model.params
r_squared = model.rsquared
print(f"Coefficients:\n{coefficients}")
print(f"R-squared: {r_squared:.3f}")
```

```
Coefficients:
const    46.666129
Age       0.672581
dtype: float64
R-squared: 0.610
```

Multiple Linear Regression Model

In [6]:
```
import pandas as pd
import numpy as np

# Set seed for reproducibility
np.random.seed(42)

# Generate random data
ages = np.random.randint(20, 61, size=20)
```

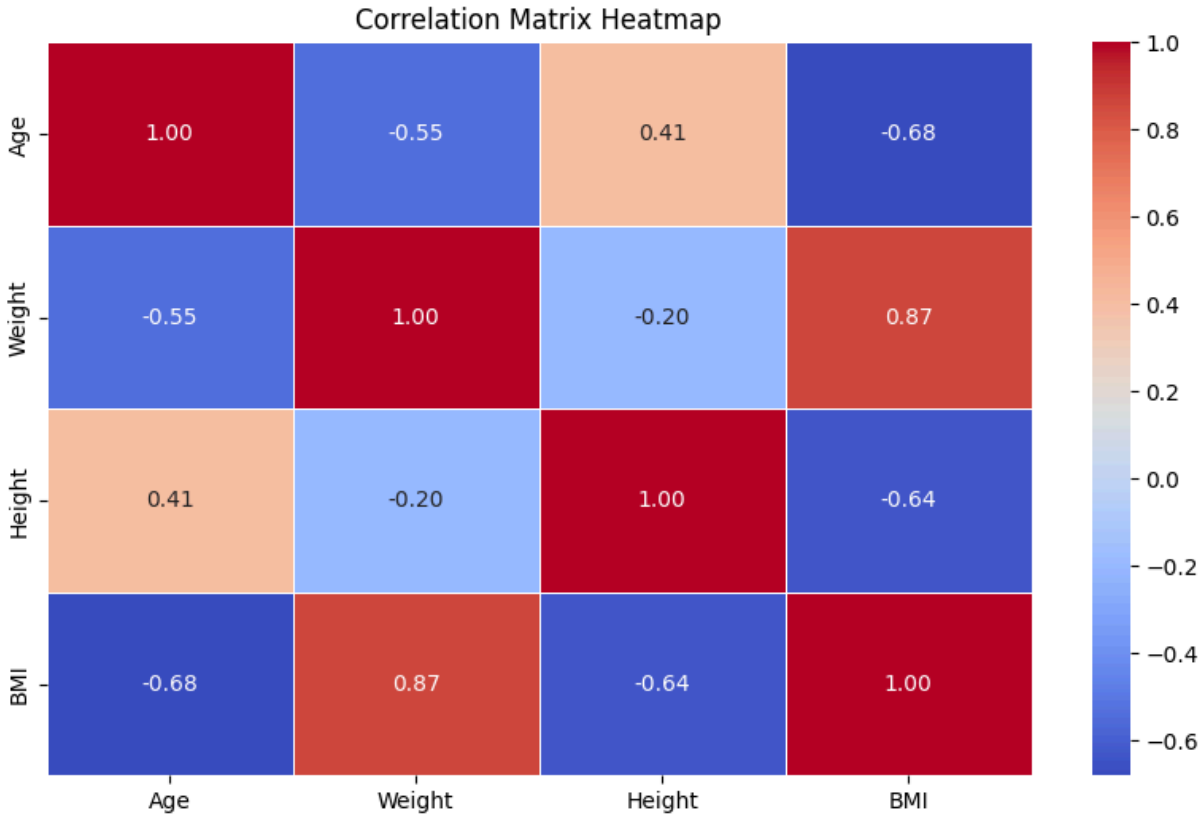
```
weights = np.random.uniform(50, 100, size=20)
heights_cm = np.random.uniform(150, 200, size=20)
heights_m = heights_cm / 100
bmi = weights / (heights_m ** 2)

# Create DataFrame
data = {
    'Age': ages,
    'Weight': weights,
    'Height': heights_cm,
    'BMI': bmi
}

df = pd.DataFrame(data)

# Display DataFrame
df
```

Out[6]:

	Age	Weight	Height	BMI
0	58	50.038938	197.110088	12.879244
1	48	99.610578	178.164411	31.380767
2	34	80.874075	169.270825	28.225725
3	27	80.582658	150.798313	35.436321
4	40	50.353315	161.544691	19.294907
5	58	51.153121	162.051273	19.479026
6	38	76.238733	184.163176	22.478642
7	42	69.993049	180.499833	21.483315
8	30	52.333283	191.659746	14.246762
9	30	98.687776	158.668233	39.199759
10	43	61.638567	169.553030	21.440821
11	55	54.530322	159.111804	21.539383
12	59	80.919300	187.768071	22.951371
13	43	69.123100	171.257794	23.567988
14	22	99.161544	160.397083	38.543429
15	41	73.338145	178.385016	23.046935
16	21	92.997020	151.565665	40.482505
17	43	84.015377	192.114239	22.763533
18	49	72.524963	172.487707	24.376493
19	57	50.663248	169.757512	17.580654

Correlation Plot: Heapmap

In [8]:
```
import seaborn as sns

plt.figure(figsize=(10, 6))
corr_matrix = df.select_dtypes(include=[np.number]).corr()
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f", linewidths=0.5)
plt.title("Correlation Matrix Heatmap")
plt.show()
```



In [9]:
```
import statsmodels.api as sm

x = df[['Age','Weight','Height']]
y = df['BMI']

x = sm.add_constant(x)
```

In [10]:
```
### Fit the Mode
model = sm.OLS(y, x).fit()
predictions = model.predict(x)

print_model = model.summary()
print(print_model)
```

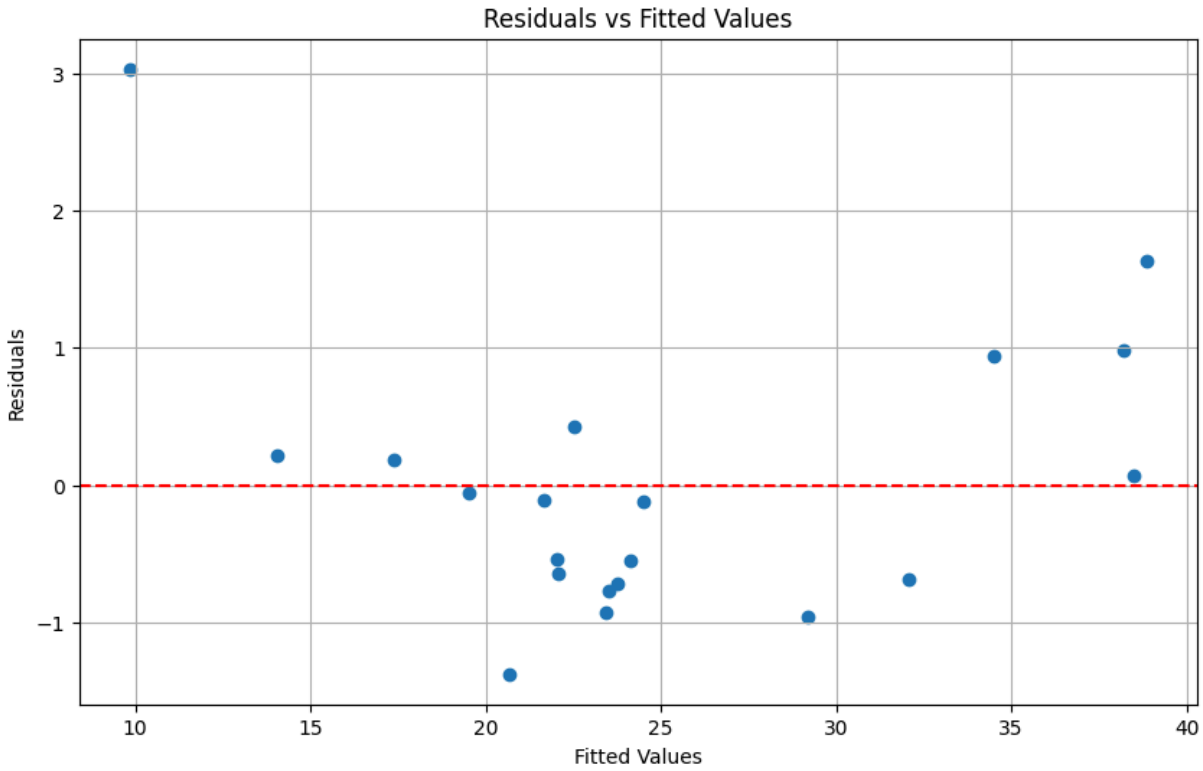
```
                            OLS Regression Results
==============================================================================
Dep. Variable:                    BMI   R-squared:                       0.983
Model:                            OLS   Adj. R-squared:                  0.980
Method:                 Least Squares   F-statistic:                     315.9
Date:                Fri, 03 Jul 2026   Prob (F-statistic):           1.91e-14
Time:                        17:06:51   Log-Likelihood:                -28.540
No. Observations:                  20   AIC:                             65.08
Df Residuals:                      16   BIC:                             69.06
Df Model:                           3
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const         49.6796      3.769     13.180      0.000      41.689      57.670
Age           -0.0706      0.028     -2.553      0.021      -0.129      -0.012
Weight         0.3325      0.018     18.636      0.000       0.295       0.370
Height        -0.2657      0.021    -12.820      0.000      -0.310      -0.222
==============================================================================
Omnibus:                       11.517   Durbin-Watson:                   2.156
Prob(Omnibus):                  0.003   Jarque-Bera (JB):                9.066
Skew:                           1.370   Prob(JB):                       0.0107
Kurtosis:                       4.837   Cond. No.                     2.87e+03
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly spe
cified.
[2] The condition number is large, 2.87e+03. This might indicate that there are
strong multicollinearity or other numerical problems.
```

Testing the Assumptions of Classical Linear Regression Models

1. A plot of Residuals Vs Fitted

In [11]:

```python
import matplotlib.pyplot as plt

# Residuals
residuals = model.resid

# Plot residuals
plt.figure(figsize=(10, 6))
plt.scatter(model.fittedvalues, residuals)
plt.axhline(y=0, color='r', linestyle='--')
plt.xlabel('Fitted Values')
plt.ylabel('Residuals')
plt.title('Residuals vs Fitted Values')
plt.grid(True)
plt.show()
```



2. Multicollinearity Check (Variance Inflation Factor - VIF):

In [12]:
```
from statsmodels.stats.outliers_influence import variance_inflation_factor

# Calculate VIF for each predictor
vif_data = pd.DataFrame()
vif_data["Variable"] = x.columns
vif_data["VIF"] = [variance_inflation_factor(x.values, i) for i in range(x.shape[1]

vif_data
```

Out[12]:

	Variable	VIF
0	const	223.688687
1	Age	1.637554
2	Weight	1.427970
3	Height	1.197290

3. Normality of Regression Residuals

In [13]:
```
Residuals = model.resid
df['Residuals'] = Residuals
df.head(5)
```

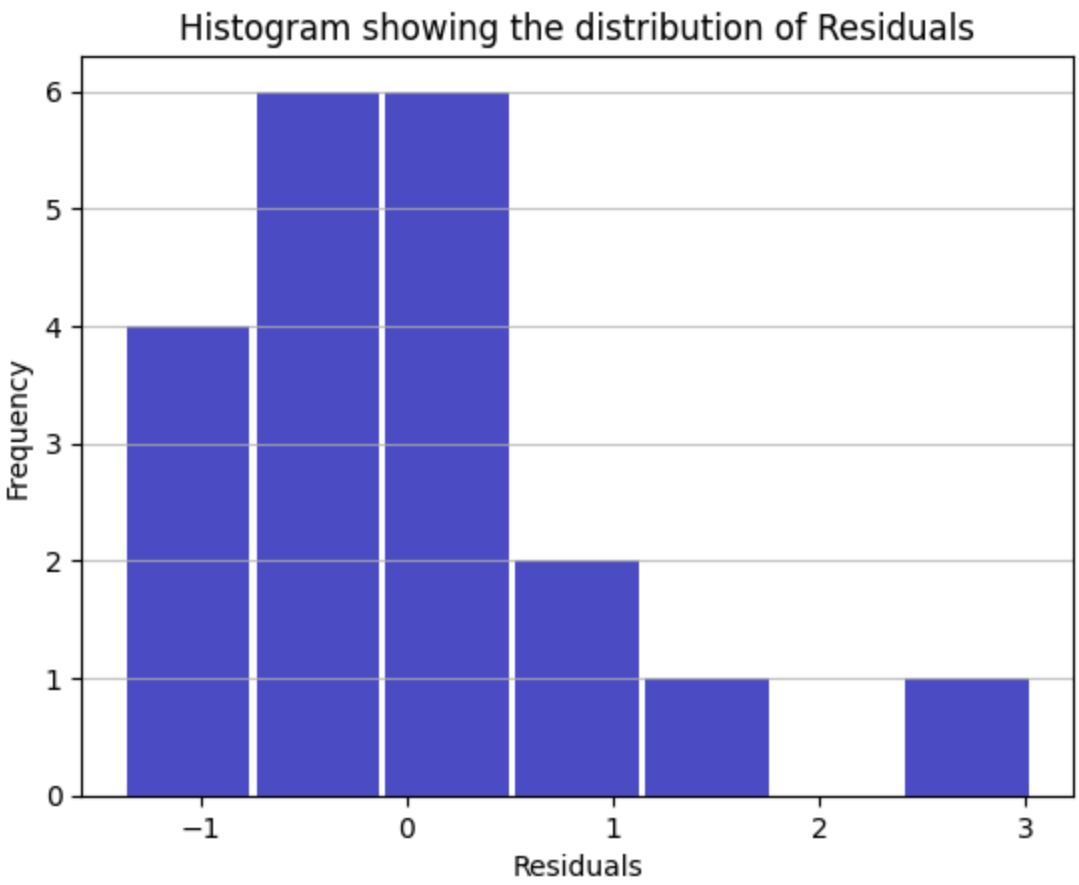
Out[13]:

	Age	Weight	Height	BMI	Residuals
0	58	50.038938	197.110088	12.879244	3.032184
1	48	99.610578	178.164411	31.380767	-0.689578
2	34	80.874075	169.270825	28.225725	-0.965767
3	27	80.582658	150.798313	35.436321	0.939046
4	40	50.353315	161.544691	19.294907	-1.377720

Histogram Showing the Distribution of the Residuals

In [14]:

```
import matplotlib.pyplot as plt
import numpy as np

# An "interface" to matplotlib.axes.Axes.hist() method
n, bins, patches = plt.hist(x=Residuals, bins='auto', color='#0504aa',
                            alpha=0.7, rwidth=0.95)
plt.grid(axis='y', alpha=0.75)
plt.xlabel('Residuals')
plt.ylabel('Frequency')
plt.title('Histogram showing the distribution of Residuals')
plt.show()
```



This assumption the mean of the regression residuals should be normally distributed with $\mu = 0$ and a constant variance σ^2

In [15]:
```
df.describe()
```

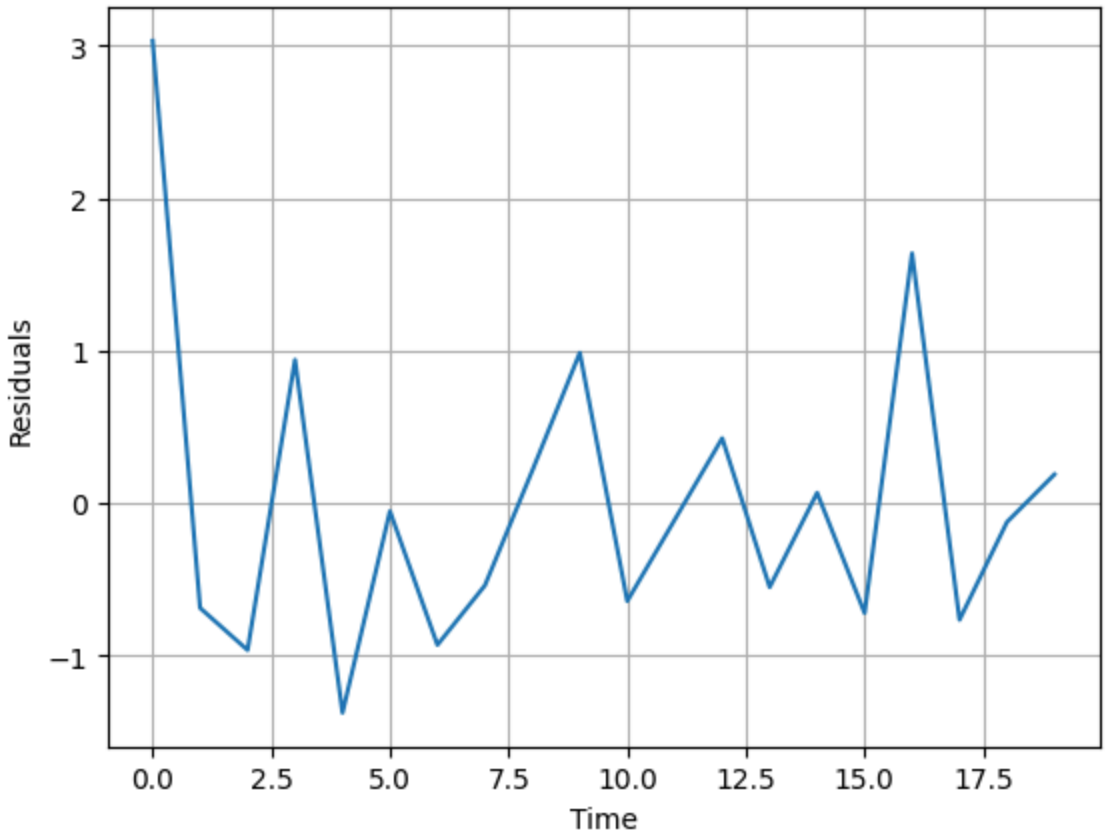
Out[15]:

	Age	Weight	Height	BMI	Residuals
count	20.000000	20.000000	20.000000	20.000000	2.000000e+01
mean	41.900000	72.438856	172.316425	25.019879	5.417888e-15
std	11.968819	17.317577	13.650293	8.027044	1.034287e+00
min	21.000000	50.038938	150.798313	12.879244	-1.377720e+00
25%	33.000000	53.981062	161.257789	20.950373	-6.979021e-01
50%	42.500000	72.931554	170.507653	22.857452	-1.177842e-01
75%	50.500000	81.693320	181.415669	29.014485	2.654996e-01
max	59.000000	99.610578	197.110088	40.482505	3.032184e+00

Plot the Residuals Over Time

In [16]:
```
import matplotlib.pyplot as plt
plt.plot(Residuals)
plt.xlabel('Time')
plt.ylabel('Residuals')
plt.grid(True)
plt.show
```

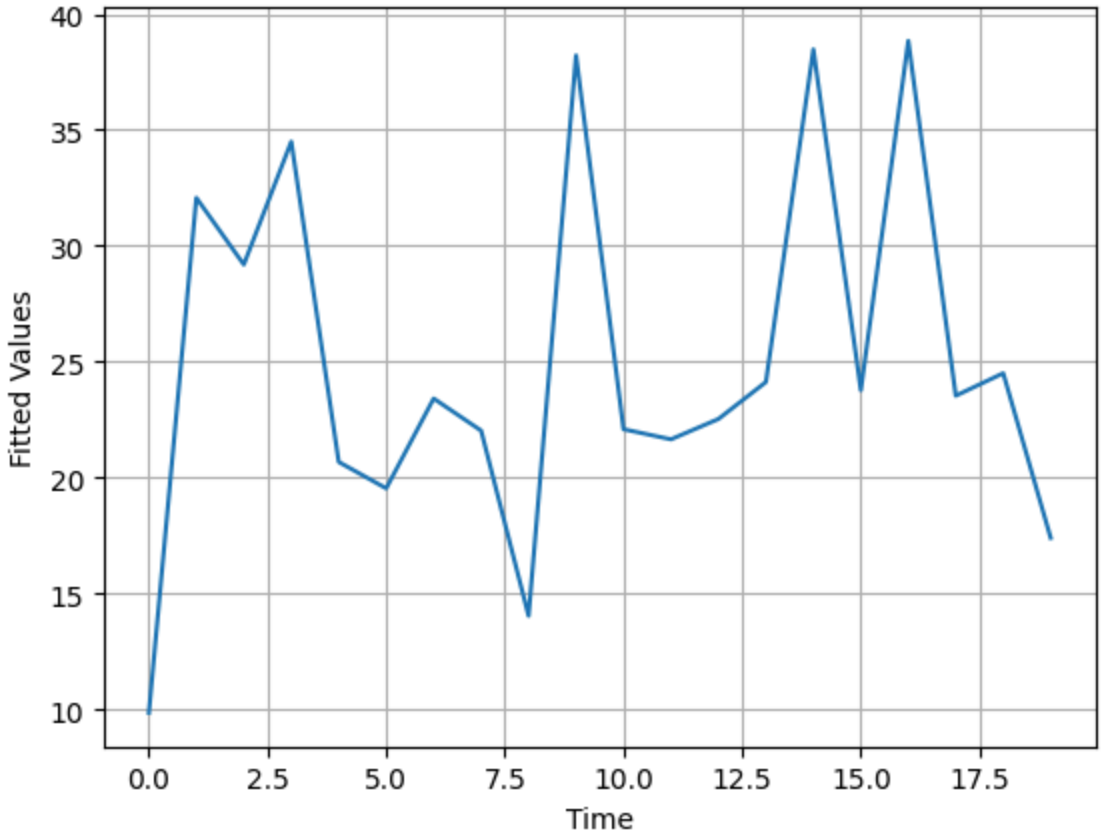
Out[16]:
```
<function matplotlib.pyplot.show(close=None, block=None)>
```



Extract the fitted values

In [17]:
```
Fitted_y = model.fittedvalues
plt.plot(Fitted_y)
plt.xlabel('Time')
plt.ylabel('Fitted Values')
plt.grid(True)
plt.show
```

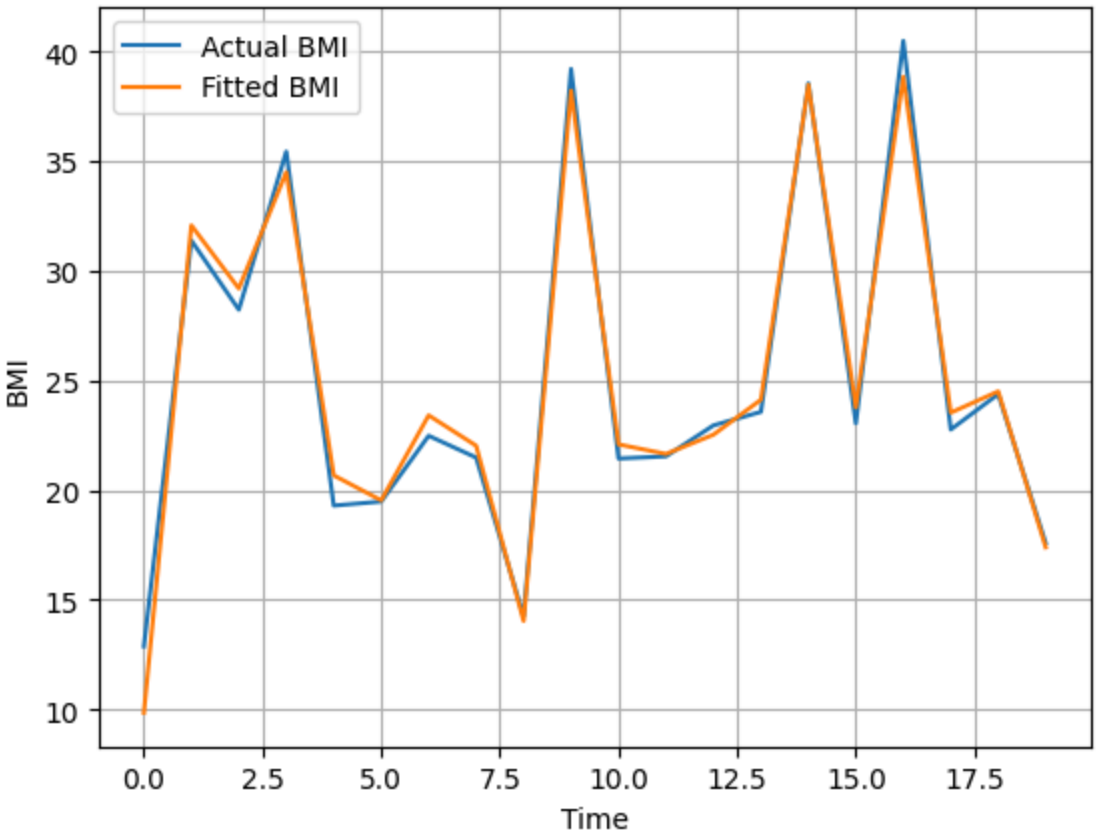
Out[17]:
```
<function matplotlib.pyplot.show(close=None, block=None)>
```



A Plot of Fitted and Actual BMI

In [18]:
```
plt.plot(df.BMI, label = "Actual BMI")
plt.plot(Fitted_y, label = "Fitted BMI")
plt.xlabel("Time")
plt.ylabel("BMI")
plt.grid(True)
legend = plt.legend()
plt.show
```

Out[18]:
```
<function matplotlib.pyplot.show(close=None, block=None)>
```



Variance covariance Assumption

This assumption requires that the covariance between the residuals and any of the predictor should be zero; $cov(x_i, \epsilon_i)=0$

In [19]:
```
cov_matrix = df.iloc[:, 1:].cov()
cov_matrix
```

Out[19]:

	Weight	Height	BMI	Residuals
Weight	2.998985e+02	-4.826449e+01	120.547717	-2.635564e-14
Height	-4.826449e+01	1.863305e+02	-70.235966	5.099671e-14
BMI	1.205477e+02	-7.023597e+01	64.433432	1.069749e+00
Residuals	-2.635564e-14	5.099671e-14	1.069749	1.069749e+00

Testing Heteroscedasticity

In [20]:
```
from sklearn.linear_model import LinearRegression
from statsmodels.stats.diagnostic import het_white
import statsmodels.api as sm
import pandas as pd

#perform White's test
white_test = het_white(model.resid,  model.model.exog)
```

define labels to use for output of White's test

In [21]:
```
labels = ['Test Statistic', 'Test Statistic p-value', 'F-Statistic', 'F-Test p-valu

#print results of White's test
dict(zip(labels, white_test))
```

Out[21]:
```
{'Test Statistic': np.float64(17.936928063320117),
 'Test Statistic p-value': np.float64(0.03591237676499789),
 'F-Statistic': np.float64(9.660312719113946),
 'F-Test p-value': np.float64(0.0007295747517714836)}
```

Here is how to interpret the output:

The test statistic is X2 = 21.666 The corresponding p-value is 0.215. White's test uses the following null and alternative hypotheses:

- Null (H0): Homoscedasticity is present (residuals are equally scattered)
- Alternative (HA): Heteroscedasticity is present (residuals are not equally scattered)

Since the p-value is not less than 0.05 we reject the null hypothesis.

In other words, we have statistically sufficient evidence to say that heteroscedasticity is present in the regress

Homework

- Analyze Relationships Between Variables :
- Use correlation analysis to measure relationships between variables in a dataset(car).
- Build simple and multiple linear regression models to analyze the impact of independent variables on a dependent variable.
- Interpret the regression coefficients, R-squared, and p-values.
- Perform model diagnostics to check assumptions and detect multicollinearity.

https://cdam.chuka.ac.ke

Thank you